Modelo de programación funcional

La programación funcional es uno de los principales paradigmas utilizados en el desarrollo de software. Su característica principal es que los programas se construyen utilizando funciones que reciben datos, realizan operaciones y producen resultados. A diferencia de otros modelos de programación, como el imperativo, la programación funcional busca reducir los cambios de estados y la modificación directa de las variables.

Este paradigma tiene sus bases en conceptos matemáticos y permite desarrollar programas más predecibles, organizados y fáciles de comprobar. Lenguajes como Haskell, lisp, scheme, Erlang y clojure utilizan ampliamente este modelo, aunque lenguajes modernos como python, JavaScript y Java también incorporan características de programación funcional.

¿Qué es la programación funcional?

La programación funcional es un paradigma de programación en el cual la solución de un problema se expresa principalmente mediante la aplicación y combinación de funciones.

Una función recibe uno o varios valores como entrada, realiza una operación genera un resultado.

Por ejemplo:

$$f(x) = x \times 2$$

Si se proporciona el valor 5:

$$f(5) = 5 \times 2 = 10$$

Se busca que una función no modifique otros elementos del programa, sino que simplemente reciba información y produzca un resultado.

Origen

La programación funcional está basada principalmente en el cálculo Lambda, desarrollado por el matemático estadounidense Alonzo Church durante la década de 1930.

El cálculo lambda fue creado como un sistema matemático para representar funciones y operaciones utilizando expresiones.

Con el paso del tiempo aparecieron otros lenguajes funcionales como:

- Lisp.
- Scheme.

- ML.
- Haskell.
- Erlang.
- clojure.

Actualmente, muchos lenguajes que originalmente no eran funcionales también incluyen elementos de este paradigma.

Uso de Funciones

Los funciones constituyen la parte principal del programa. cada función recibe determinados datos y produce un resultado.

Ejemplo:

```
Suma a b = a + b
```

si se ejecuta:

```
Suma 4 6
```

el resultado es:

```
10
```

Funciones puras

Una función pura siempre devuelve el mismo resultado cuando recibe los mismos valores.

Por ejemplo:

```
Cuadrago x = x * x
```

Si se ejecuta:

```
Cuadrado 4
```

Siempre devolvera:

```
16
```

Inmutabilidad

Significa que los datos no se modifican directamente. En lugar de cambiar un valor existente, se genera uno nuevo.

Recursividad

La recursividad ocurre cuando una funsion se llama a sí misma para resolver un problema.

Ejemplo:

```
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

Funciones de orden Superior

Son funciones que pueden recibir otras funciones como parámetros o devolver una función como resultado.

Composición de funciones

Permite combinar varias funciones pequeñas para realizar operaciones más complejas.

Funciones Comunes

En programación funcional se utilizan operaciones como:

- Map: aplica una función a todos los elementos de una lista.
- Filter: Selecciona elementos que cumplen una condición.
- Reduce o fold: Combina varios elementos para obtener un único resultado.

Ejemplo en Haskell:

```
map (*2) [1,2,3,4]
```

Resultado =

[2, 4, 6, 8]

Lenguajes de programación funcional

Algunos de los lenguajes más relacionados con este paradigma son:

- Haskell.
- Lisp.
- Scheme.
- Erlang.
- Clojure.
- F#.

También existen lenguajes multiparadigma que permiten trabajar con programación funcional, como Python, JavaScript, Java, Scala.

Calculo lambda de Alonso crunch

Alonso Church desarrollo el calculo lamba en la decada de 1930 como un sistema formal para investigar los fundamentos de la matemática y la computabilidad. Se convirtió en uno de los tres modelos formales equivalentes de computación y hoy es la base teórica de los lenguajes de programación funcional.

El cálculo lambda es un sistema minimalista donde todo es una función. No tiene números, booleanos ni estados; si se necesitan, se codifican como funciones. Su sintaxis se construyen con solo 3 reglas inductivas:

- Variables: Cualquier símbolo como x es un término
- Abstracción: Si E es un término y x una variable, λx. E es una función (el λx es el parámetro, E el cuerpo).
- Aplicación: Si E y F son términos, EF es la aplicacion

Como funciona: La reduccion β

El cálculo se ejecuta mediante β-reducción, que es Sustitución★

C.lang

Es un lenguaje de programacion de propósito general creado originalmente en el laboratorio de ciencias de la computacion de Ericsson. Fue diseñado principalmente para desarrollar sistemas que necesitan funcionar continuamente, manejar muchas operaciones al mismo tiempo y recuperarse de errores sin detener todo el sistema. Erlang es considerado principalmente un lenguaje de programacion funcional y destacan especialmente por su capacidad para trabajar con concurrencia sistemas distribuidos y tolerancia a fallos.

Caracteristicas

Concurrencia: permite ejecutar una gran cantidad del procesos al mismo tiempo.

Paso de mensaje: los procesos se comunican enviandose informacion entre ellos.

Tolerancia a fallos: si un proceso falla, el resto del sistema puede continuar funcionando.

Erlang se utilica especialmente en telecomunicaciones, servidores, comercio electrico, comercio, sistemas bancarios.

Ericsson, por ejemplo, lo han utilizado en infraestructura de telecomunicaciones.

Ejemplo

```
-module (holu)
-export ([Saludar/0]).

Saludar () ->
        io: format ("Holu mundo \n")
```

Este programa crea un modulo llamado holu y una funcion llamada saludo, que imprime Holu Mundo.

Elixir

Una caracteristicas muy importante es que Elixir se ejecuta sobre la maquina virtual de Erlang, conocida como BEAM. Por eso puede aprovechar muchas de las caracteristicas que hicieron famosos Erlang, como la concurrencia, escalabilidad y tolerancia a fallos. Elixir fue diseñado buscando que el desarrollo de aplicaciones fuera más productivos y que el código fuera fácil de mantener, sin perder las capacidades de Erlang.

Caracteristicas

* Es un lenguaje funcional
* Utilica la maquina muchos procesos ligeros
* Ofrece concurrencia
* Tiene de mecanismos de tolerancia a fallos
* facilita la construccion de sistemas distribuidos

Ejemplo

```
defmodule Hola do
  def saludar do
    IO.puts("Hola mundo")
  end
end
```

Despues de ejecutarse

```
Hola.saludar()
```

y mostrara

Hola mundo

Donde utilica Erlang y Elixir en WhatsApp?

WhatsApp ha utilizado principalmente Erlang en el backend de sus servidores, ya que permite manejar una gran cantidad de usuarios y mensajes al mismo tiempo.

Erlang se utiliza para:

- Mantener muchas conexiones simultáneas
- Procesar y entregar mensajes
- Manejar notificaciones
- Evitar que un fallo afecte todo el sistema
- Permite que la plataforma pueda crecer y atender millones de usuarios

Elixir, aunque también funciona sobre maquinas virtual BEAM de Erlang, no esta documentado como una tecnologia principal de WhatsApp.

WhatsApp es un ejemplo importante del uso de Erlang para sistemas de mensajeria a gran escala, Elixir comparte su tecnologia base, pero no se puede afirmar que WhatsApp lo utilice directamente.

Características	Funciones prefijas	Funciones infijas	Funciones postfijas.
Definición	la función se escribe antes de sus argumentos.	la función u operador se coloca entre dos argumentos.	La función se coloca después de sus argumentos.
Sintaxis	función Arg1 Arg2	Arg1 operador Arg2	Arg función
orden	Función → Argumento	Argumento → Función → Argumento	Argumento → función
Ejemplo	Max 5 8 R= 8	5 + 8 R=13	5 factorial (No válido)
Uso en Haskell	Es la forma habitual de llamar función	Se utiliza principalmente con operadores y funciones entre acentos graves.	No existe como forma general de aplicaciones de funciones
Ventajas	Facilita la aplicación de funciones y permite varios argumentos	Hace más legibles las operaciones matemáticas	No tiene soporte en la sintaxis de Haskell.

Investigación U1

¿Qué son los intervalos?

Los intervalos en Haskell son una forma de generar secuencias de valores automáticamente, sin necesidad de escribir cada elemento por separado.

Se utilizan principalmente para crear listas de números o caracteres que siguen un orden determinado.

Tipos de intervalo:

1: Intervalo Simple: Genera una lista de valores desde un número inicial hasta uno final

2: Intervalo con incremento: Permite establecer cuánto aumentará cada número de la lista.

3: Intervalo descendentes: Genera valores de mayor a menor utilizando un incremento negativo.

4: Intervalo de caracteres: Permite generar una secuencia de letras

5: Intervalo infinito: Genera una secuencia que no tiene un límite definido

Los intervalos permiten crear listas rápidamente, trabajan con secuencias numéricas, recorren valores y reducir la cantidad de código necesario.

¿Que son los operadores?

Los operadores son símbolos que permiten realizar operaciones matematicas, comparaciones y evaluaciones lógicas entre diferentes valores

En Haskell existen varios tipos de operadores

Operadores Aritmeticos:	Ejemplo:
Sumas	5 + 3
Restas	10 - 4
Multiplicaciones	4 * 3
Division fraccionarios	10 / 2
Division enteros	10 div 3
Residuos de division	10 mod 3
Potencia	2 ^ 3

Operadores de Comparación:	Ejemplo
igual que	5 == 5
Diferente de	5 /= 3
Mayor que	8 > 4
Menor que	3 < 7
Mayor o igual	5 >= 5
Menor o igual	4 <= 6

Operadores Logicos:	Ejemplo
Y lógico (AND)	True && false
O lógico (OR)	True || Fabe
Negación	not true

Los operadores nos permiten realizar cálculos, comparar información evaluar condiciones y manipular listas dento de un Programa

Evaluacion perezosa en Haskell

La evaluacion significa que solamente realiza un calculo cuando necesita conocer su resultado. Es decir no realiza todas las operaciones desde el principio solo calcula lo que se necesita en ese momento.

ejemplo :

Tenemos una lista de numero de 1 a 1000, pero solo queremos los primeros 5 numeros.

Take 5 [1 .... 1,000]

Resultado = [1, 2, 3, 4, 5]

Haskell solamente necesita generar los primeros 5 numeros, porque solo eso se solicitaron.
No fue necesario generar los 1000 numeros completos.